

VIDEO NOTES · PART 4

Prompts in LangChain

Templates, chat messages, and learning from examples

What this document is

A plain-English guide to the Prompts component —
how to write prompts that are reusable, structured and reliable.

1 Why Prompts Need Structure

A prompt is just the instruction you send to the model. Writing one is easy. Managing hundreds of them across a real application is not.

The moment the same instruction appears in several files with slightly different wording, you lose the ability to improve it — you can no longer tell which version produced which result.

The problem this component solves: prompts written inline are copied, drift apart, and cannot be tested or versioned. Prompt templates pull them into one place with clearly marked blanks.

LangChain offers two template types, and choosing the right one is the first decision you make.

2 Two Kinds of Template

PromptTemplate

For **inference** work — one instruction in, one answer out.

No conversation. No history.

Summarise, classify, extract, rewrite

ChatPromptTemplate

For **chat** applications — a back-and-forth made of separate messages.

Has roles and keeps context.

Assistants, support bots, copilots

Figure 1 — Pick by the shape of the interaction, not by the task's difficulty.

Question	PromptTemplate	ChatPromptTemplate
Shape	A single block of text	A list of messages
Roles	None	System, Human, AI
Conversation	One-off request	Ongoing exchange
Typical use	Batch processing, pipelines	Anything a user talks to

3 PromptTemplate — Reusable Instructions

A prompt template is your instruction written once, with named blanks where the details go. You supply values for those blanks each time you use it.

THE TEMPLATE — WRITTEN ONCE

You are a **profession**.
Explain **topic** to a general audience.
Keep your answer under **word_limit** words.

Values supplied

profession, topic, word_limit



Blanks filled in



Final prompt

sent to the model

Figure 2 — One template serves every combination of profession, topic and length.

The same three blanks give you a doctor explaining diabetes in 50 words, or a lawyer explaining contracts in 200 — without a second copy of the instruction existing anywhere.

What you gain

Benefit	Why it matters
One source of truth	Improve the wording once and every caller benefits immediately.
Fewer mistakes	The template declares which variables it needs, so a missing value is caught early rather than producing a broken prompt.
Versionable	Because the prompt is a real file, changes to it show up in your version history like any other code change.
Testable	You can run the same template against many inputs and compare results properly.

4 ChatPromptTemplate — Working With Roles

Chat models do not see one block of text. They see a sequence of **messages**, each labelled with who said it. Those labels change how the model responds.

SYSTEM You are a polite support agent for an online store. Never promise a refund without checking the order.

HUMAN My order arrived damaged.

AI I am sorry to hear that. Could you share your order number?

HUMAN It is 48812.

Figure 3 — A chat prompt is a stack of role-labelled messages, not one paragraph.

Role	Its job
System	Sets the behaviour, tone and rules. Written by you, not the user, and it applies to the whole conversation.
Human	What the user actually typed.
AI	What the model replied earlier. Including it is what gives the conversation continuity.

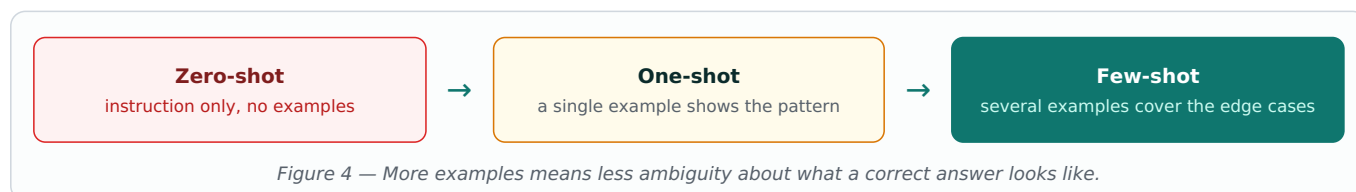
The system message is where most behaviour lives. Rules, tone, what the bot must refuse, what format it should answer in — all of it belongs there, stated once, rather than repeated in every user message.

5 Teaching by Example: Shots

A model knows a great deal about the world in general. It knows nothing about how *your* organisation labels things.

Ask it to sort tickets into your internal categories and it will invent reasonable labels — just not yours. The fix is not a longer explanation. It is examples.

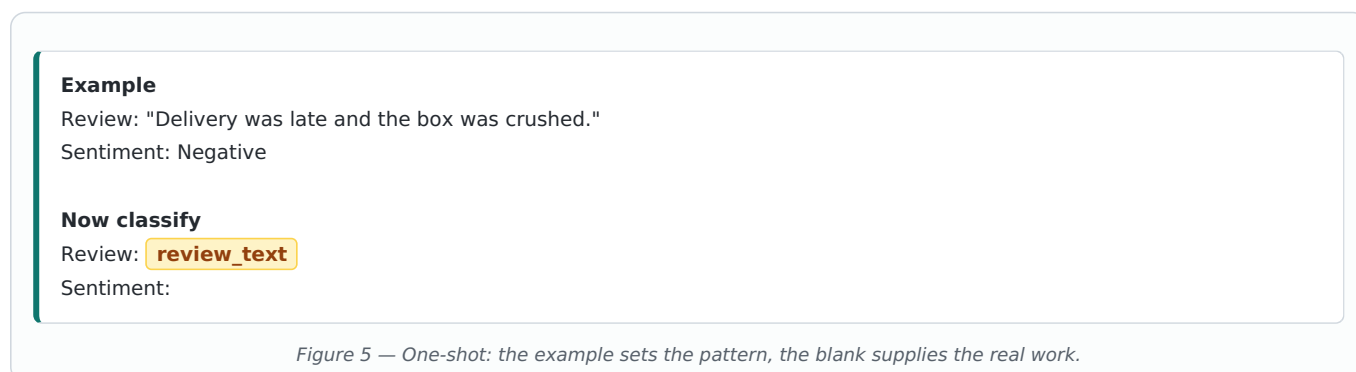
Each example you include is called a **shot**.



The key insight: examples do not teach the model new knowledge. They demonstrate the exact *format* and *labels* you expect — which is usually what was actually missing.

One-shot — sentiment analysis

With a single worked example, the model sees the expected input, the expected output, and the shape of the answer. It then follows that pattern.



Notice the trailing label with nothing after it. That is deliberate — the model completes the pattern rather than writing a paragraph around it.

Few-shot — ticket categorisation

One example is enough when categories are obvious. Real category sets are not obvious. When several labels could plausibly apply, you need examples that show where the boundaries sit.

Ticket: "I was charged twice this month." → **Billing**
Ticket: "The app crashes when I open settings." → **Technical**
Ticket: "How do I change my delivery address?" → **Account**
Ticket: "My refund has not arrived yet." → **Billing**

Ticket: **new_ticket** →

Figure 6 — Few-shot: several examples map out the category boundaries, including a second Billing case that could have been mistaken for something else.

Why this is an industry-standard technique

The example given in the lecture is **procedure code mapping** in healthcare — matching messy, free-text descriptions of a medical procedure to a strict internal code.

No general model knows your code list. Explaining it in prose would be enormous. A handful of well-chosen examples communicates it far more precisely.

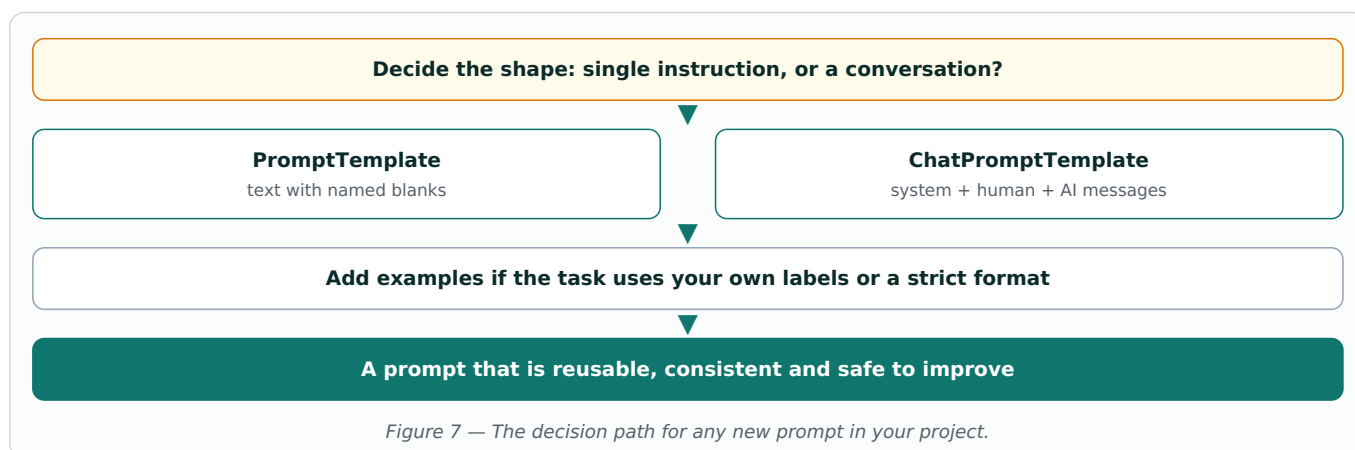
Where few-shot fits	Why examples beat explanation
Organisation-specific labels	The categories exist only inside your company; nothing in training data covers them.
Routing and triage	Sending a request to the right team depends on internal conventions, not general logic.
Code and ID mapping	Free text to a fixed code list, where exactness matters and near enough is wrong.
Strict output formats	An example of the format is clearer and shorter than describing it.

Choosing how many examples

Use	When	Watch out for
Zero-shot	The task is common and the output shape is obvious	Inconsistent formatting between runs
One-shot	You mainly need to fix the output format	The model over-fitting to that one example's style
Few-shot	Categories are yours, or cases are easily confused	Longer prompts cost more and run slower

Practical rule: pick examples that sit near the boundaries, not the easy middle. Two confusable cases teach more than ten obvious ones.

6 Putting It Together



7 Coming Next

The lecture ends by naming four topics that follow.

Topic	What it is, briefly
Chain of Thought prompting	Asking the model to reason step by step before answering, which improves accuracy on problems that need working out rather than recall.
Partial prompting	Filling some template blanks in advance and leaving the rest for later — useful when part of the context is known up front.
Message placeholders	A reserved slot inside a chat template where an entire conversation history can be dropped in. This is the bridge to the memory component.
Output Parsers	Converting the model's reply into structured data — the natural next step once prompts are producing consistent output.

Practice before the next lecture

- ☐ Write one PromptTemplate with at least three named blanks and reuse it twice.
- ☐ Build a ChatPromptTemplate and change only the system message — observe the difference.
- ☐ Take a task that failed zero-shot and add one example to it.
- ☐ Write a few-shot prompt for a category set from your own work.
- ☐ Deliberately include one confusable example and check it helps.
- ☐ Store your prompts in the *prompts* folder from the setup lecture.

The short version: use PromptTemplate for one-off inference and ChatPromptTemplate for conversations, put your rules in the system message, and when the model does not know your labels, show it examples rather than writing longer instructions.